AI Hallucination Detector

Team Roles & Work Boundaries — who builds what, what each module hands to the next, and the rules that keep everyone's work from overlapping.
Repo: github.com/abhishek-the-goat/ai-hallucination-detector

How the system fits together

A user pastes an AI-generated answer into the dashboard. The **Claim Extractor** breaks it into single checkable facts. The **Retrieval Engine** finds evidence for each fact in a trusted corpus. The **Verification Engine** scores each fact as supported, contradicted, or unverified. The **Dashboard** shows the final report with the made-up lines highlighted.

One rule governs everything: **each person owns exactly one folder and codes only inside it.** Modules talk to each other only through the fixed contracts below — never by editing another person's files.

The shared contracts (fixed)

Every module connects through these data shapes and functions. They are fixed for the whole project. If a contract genuinely has to change, it is decided in the group first and this document is updated in the same pull request — nobody changes a shared function silently.

Data shapes

```
Claim = {"claim_id": int, "text": str}
Evidence = {"text": str, "source_title": str, "source_url": str, "score": float}
Verdict = {"claim_id": int, "label": "SUPPORTED" | "CONTRADICTED" | "UNVERIFIED",
"confidence": float (0 to 1), "evidence": [Evidence, ...]}
Report = {"verdicts": [Verdict, ...], "hallucination_score": float}
```

Functions

Function	Built by	What it does
`extract_claims(answer_text) -> list[Claim]`	Pratham	Splits a full AI answer into single checkable facts
`retrieve_evidence(claim_text, k=5) -> list[Evidence]`	Ishan	Returns the top-k evidence passages for one claim
`verify_claim(claim, evidence) -> Verdict`	Abhishek	Scores one claim against its evidence
`run_pipeline(answer_text) -> Report`	Abhishek	The single entry point — strings all modules together; the dashboard calls only this

Abhishek — Verification Engine + Integration

Owns the folder: `verification/`

Your job

- Build `verify_claim()`: run an NLI (natural language inference) model on each claim–evidence pair to get entailment / contradiction / neutral scores.
- Combine the NLI result with embedding similarity into one verdict and a confidence score between 0 and 1.
- Apply the verdict rules: SUPPORTED when entailment is the strongest signal and above threshold; CONTRADICTED when contradiction is strongest; UNVERIFIED otherwise.
- Build `run_pipeline()`: take the raw answer text, call the extractor, then retrieval per claim, then verification, and assemble the final Report with the overall hallucination score.
- Integration: maintain the repo structure, `requirements.txt`, README, and run the final end-to-end tests.
- **Only you merge pull requests.** Teammates open PRs on their own branches; you review them, confirm the CI check is green, and merge. Nobody else merges into `main`.

You receive / you hand over

- **Receives:** Claims from Pratham's `extract_claims()`; evidence lists from Ishan's `retrieve_evidence()`.
- **Hands over:** The finished Report (verdicts + hallucination score) to Divyansh's dashboard, via `run_pipeline()`.

Your files

```
verification/nli.py — NLI model loading + scoring
verification/scoring.py — verdict rules + confidence
verification/pipeline.py — run_pipeline()
tests/ — verdict rule tests
```

Tools: transformers (an MNLI model such as DeBERTa), sentence-transformers, pytest

Done means: The pipeline runs end-to-end on a sample answer and returns verdicts plus a hallucination score; verdict unit tests pass in CI.

Stay out of: Corpus building and indexing, claim-splitting logic, and all dashboard pages.

Ishan — Retrieval Engine

Owns the folder: `retrieval/`

Your job

- Build the trusted corpus: choose the source (a Wikipedia subset or the FEVER dataset), download it, and clean the text.
- Chunk the corpus into passages of roughly 200–300 words, each stored with its title and source URL.
- Build the keyword index: an inverted index with BM25 ranking (rank-bm25 library).
- Build the meaning index: embed every chunk with sentence-transformers and store the vectors in FAISS.
- Build `retrieve_evidence()`: hybrid search — merge BM25 results and vector-search results, deduplicate, and return the top-k passages as Evidence objects.

You receive / you hand over

- **Receives:** One claim at a time, as a plain text string, from the pipeline.
- **Hands over:** A list of Evidence objects (passage text, source title, source URL, score) for each claim, to Abhishek's verification engine.

Your files

```
retrieval/corpus.py — download + chunk the corpus
retrieval/index.py — BM25 index + FAISS index
retrieval/search.py — retrieve_evidence()
```

Tools: rank-bm25, sentence-transformers, FAISS, Wikipedia / FEVER data

Done means: Given any claim string, the function returns 5 relevant passages with working source links, fast (about 2 seconds or less), and the whole index rebuilds from scratch with one command.

Stay out of: NLI and verdict logic, claim splitting, and the dashboard.

Pratham — Claim Extractor + AI Input

Owns the folder: `claim_extractor/`

Your job

- Build `extract_claims()`: take a full AI answer and return a list of single, standalone factual claims.
- Split the answer into sentences, then split compound sentences so each claim contains exactly one fact.
- Filter out non-claims: opinions ("I think…"), questions, greetings, and recommendations are removed — only factual statements become claims.
- Decontextualize each claim: replace pronouns and vague references with the actual entity ("He was born in 1961" becomes "Barack Obama was born in 1961") so the claim can be checked on its own.
- Build the AI input side: the code that obtains the answer to be tested — from an AI model API or a local Ollama model — and passes its text into the pipeline.

You receive / you hand over

- **Receives:** Raw answer text — pasted by the user through the dashboard, or generated by the AI model call.
- **Hands over:** A list of Claim objects (claim_id + clean standalone text) to the pipeline.

Your files

```
claim_extractor/extractor.py — extract_claims()
claim_extractor/filters.py — opinion / question filter
claim_extractor/ai_input.py — AI model call
```

Tools: spaCy, requests, an LLM API or local Ollama (optional, for extraction help)

Done means: A 5-sentence answer comes out as clean, standalone factual claims, one fact each, with opinions and questions filtered out.

Stay out of: Evidence search, verdict logic, and the dashboard UI.

Divyansh — Dashboard Website

Owns the folder: `dashboard/`

Your job

- Build the Flask web app: a home page with one text box — paste an AI answer (or type a question) and press Check.
- Build the results page: every claim shown in place, highlighted by verdict — green = supported, red = contradicted, yellow = unverified — with the confidence percentage next to it.
- Build the evidence panel: clicking any claim opens the source passages that support or contradict it, with clickable source links.
- Show the summary header: the overall hallucination score (percentage of claims not supported).
- Build the history page: past checks saved in a database (SQLite) and listed for review.
- The dashboard calls only `run_pipeline()` and renders its Report. No ML or search code lives in the dashboard.

You receive / you hand over

- **Receives:** The Report JSON from Abhishek's `run_pipeline()`.
- **Hands over:** The working website the team presents and demos.

Your files

```
dashboard/app.py — Flask routes
dashboard/templates/ — pages (home, results, history)
dashboard/static/ — CSS / JS
dashboard/db.py — saved check history
```

Tools: Flask, HTML/CSS/JavaScript, SQLite

Done means: Paste an answer, press Check, and the highlighted report renders with working evidence links and a correct hallucination score.

Stay out of: Extractor, retrieval, and verification code. If the Report shape does not fit the UI, raise it in the group — do not edit pipeline code.

Anti-overlap rules (AntiGravity)

- **One folder per person.** Point AntiGravity only at your own folder. Its edits stay inside your module.
- **Never regenerate another module's file.** If someone else's output breaks your code, the shared contract is the issue — raise it in the group instead of patching their file.
- **Contracts are fixed.** Function names, argument shapes, and JSON fields in this document do not change without a group decision and a matching update to this document.
- **One branch per person:** `feat/claim-extractor`, `feat/retrieval`, `feat/verification`, `feat/dashboard`. Work flows: branch → push → pull request → CI passes → merge.
- **Abhishek merges every pull request.** Teammates open PRs and hand them over; merging into `main` is done only by Abhishek, after the CI check passes.
- **Shared files need a heads-up.** `requirements.txt` and `README.md` belong to everyone — announce in the group before editing them.
- **No duplicates.** If two modules need the same helper (for example text cleaning), it lives in exactly one module and the other imports it. Never copy-paste the same code into two folders.
- **main is protected.** Direct pushes are blocked for everyone. If the CI check fails, the pull request cannot merge — fix the code, not the check.

At a glance

Person	Module	Delivers
`Abhishek`	verification/	Verdicts, confidence scores, the working end-to-end pipeline
`Ishan`	retrieval/	Evidence passages for any claim, with sources
`Pratham`	claim_extractor/	Clean standalone claims from any AI answer
`Divyansh`	dashboard/	The website: check page, highlighted report, history